

Normalization

Internal Covariate Shift

The internal covariate shift problem in deep learning refers to the change in the distribution of network activations due to updates in the network parameters during training, causing the layers to continuously adapt to new distributions.

Internal Covariate Shift

Example:

Suppose a hidden layer initially receives: $x = [2, 4, 6, 8]$

Mean: $\mu = 5$

After weight update in previous layer, let new inputs become: $x = [20, 40, 60, 80]$

Mean: $\mu = 50$

- The distribution has shifted drastically.
- The next layer now sees completely different scale and statistics.
- This is internal covariate shift.

Internal Covariate Shift

Internal covariate shift results in the following problems:

- Slow learning/ converging rates during training
- Risk of vanishing/exploding gradients
- Overfitting problem
- Poor Generalization: Not perform well on unseen data

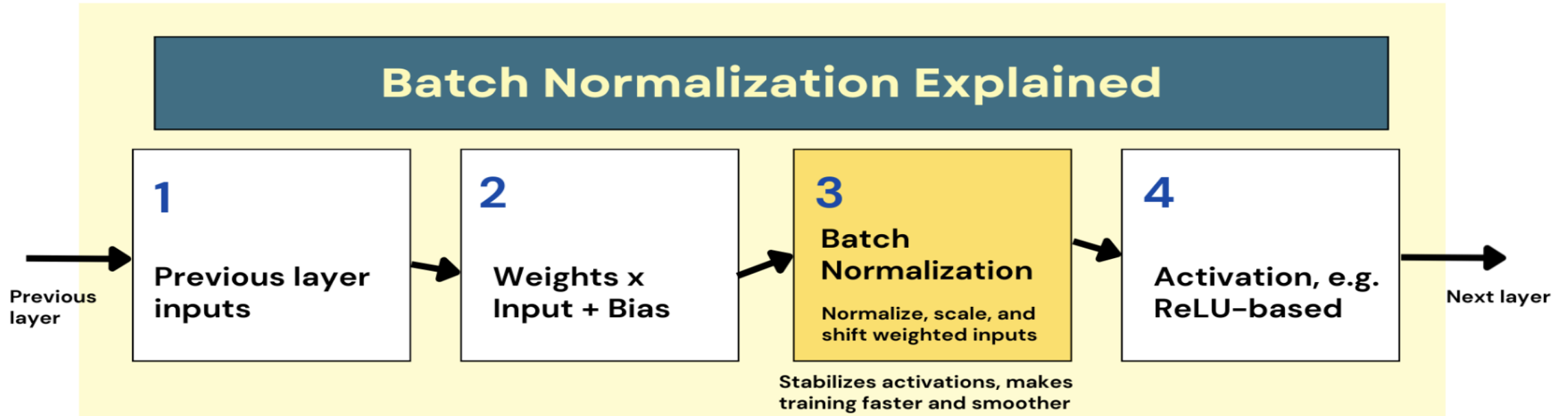
Batch Normalization

- Batch Normalization (BatchNorm) is a technique that normalizes the inputs to layers in a neural network to improve the training process.

OR

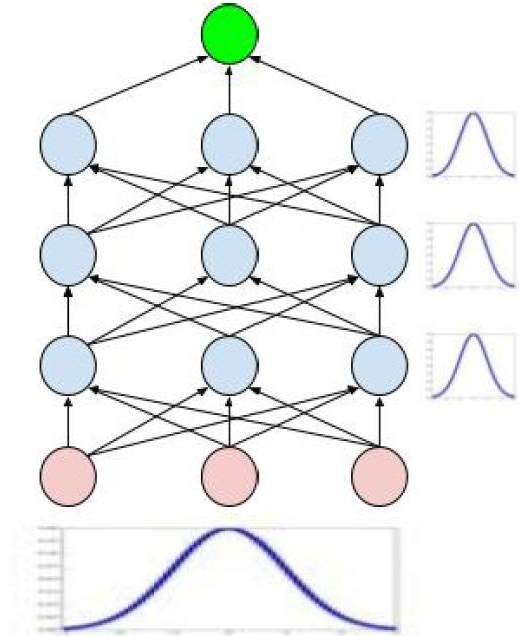
- Batch Normalization is used to reduce the problem of internal covariate shift in neural networks.
- It works by normalizing the data within each mini-batch. How?
 - It calculates the mean and variance of data in a batch and then adjusts the values so that they have similar range (step 1).
 - After that it scales and shifts the values so that model learn effectively (step 2).

Batch Normalization



BatchNorm operates by standardizing the input values to a layer so that they have a mean of zero and a standard deviation of one within each mini-batch.

Batch Normalization



- It would help the network if the pre-activations at each layer were unit gaussians, i.e.;
 - Mean = 0
 - Variance = 1
- It is ensured explicitly by using the batch normalization

Mathematical functions of BatchNorm

$$\mu = \frac{1}{m} \sum_{i=1}^m x_i$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2$$

$$y_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

Where:

μ = mean for the mini-batch

σ^2 = variance for the mini-batch

m = input size

x_i = i^{th} dimension of input x

y_i = i^{th} dimension of input y

γ = scale, learnable parameters

β = shift, learnable parameters

ϵ = small constant to avoid division by zero

Example: Training WITHOUT Batch Normalization

Mini-batch input: $x = [10, 20, 30, 40]$

Weight: $W = 5$

- Distribution shifted significantly
- Next layer must re-adapt

Forward Pass: $z = Wx = [50, 100, 150, 200]$

Mean: $\mu = 125$

Note: Large activations $\rightarrow$ next layer receives large values.

After Weight Update:

$$W = 8$$

$$z = [80, 160, 240, 320]$$

Now mean: $\mu = 200$

Example: Training WITH Batch Normalization

Same mini-batch input $z = Wx = [50, 100, 150, 200]$

Step 1: Mean: $\mu = 125$

Step 2: Variance $\sigma^2 = 3125$

$$\sigma = 55.9$$

Step 3: Normalize $\hat{z} = \frac{z - 125}{55.9}$

$$\hat{z} \approx [-1.34, -0.45, 0.45, 1.34]$$

Note: Now, mean ≈ 0 , variance ≈ 1

After Weight Update:

$$W = 8$$

$$z = [80, 160, 240, 320]$$

After BatchNorm:

$$\hat{z} \approx [-1.34, -0.45, 0.45, 1.34]$$

- Distribution remains stable.
- Next layer sees consistent inputs.
- Faster & stable training.

Example: Training WITH Batch Normalization

If we use Scale and Shift (step 2)

Suppose: $\gamma = 2, \beta = 1$

Now: $y = \gamma \hat{z} + \beta$

$$y = 2\hat{z} + 1$$

$$y \approx [-1.68, 0.10, 1.90, 3.68]$$

Why scale (γ) and shift (β)?

- These parameters allow the model to learn the optimal scaling and shifting of the normalized activations giving the network additional flexibility.
 - Keep normalized outputs (if optimal)
 - Or undo normalization completely (if needed)
- Because of scale and shift the network learn itself what is best for it
- γ and β are learnable parameters, where γ is used for scaling and β is used for shifting.
- In Keras, the initial values are $\gamma = 1$ and $\beta = 0$. These values are learned during training through backpropagation.

Batch Normalization

features 

samples 

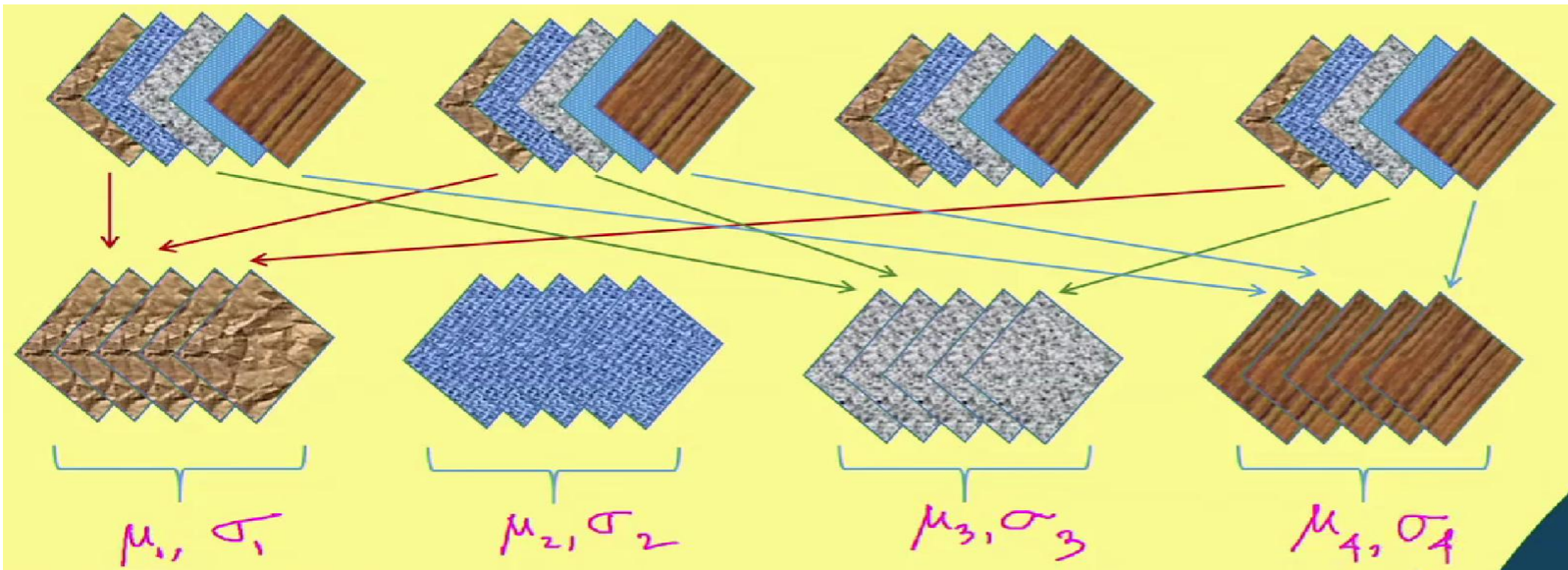
	Feature 1	Feature 2	Feature 3
Sample 1	6	3	9
Sample 2	7	2	4
Sample 3	8	1	5
Mean	7	2	6
Std.	0.82	0.81	2.16

Normalized features 

	Feature 1	Feature 2	Feature 3
Sample 1	-1.31	5.33	8.12
Sample 2	2	2	-2.08
Sample 3	5.31	-1.33	-0.04

Batch Normalization: Advantage

- Stable: Can set the value of hyperparameters in wider range
- Faster: Can set higher learning rate
- Reduce the impact of negative weight initialization



Layer Normalization

- Layer Normalization (LN) is a normalization technique used in deep neural networks where normalization is performed across the features of a single data sample, instead of across the batch.
- Mathematical function will remain the same as in Batch Normalization
 - Compute Mean and Variance: For each data sample, calculate the mean and variance across all features.

$$\mu = \frac{1}{m} \sum_{i=1}^m x_i$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2$$

Layer Normalization

- Normalize: Subtract the mean and divide by the square root of the variance (with a small for stability).
- Scale and Shift: Apply learnable parameters (scale) and (shift) to the normalized values.

$$y_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

- Preferred for sequential models like recurrent neural networks (RNNs).
- Widely adopted in transformer-based architectures, such as BERT and GPT.
- Ideal for scenarios where batch size is small or varies dynamically.

Layer Normalization

features →

samples ↓

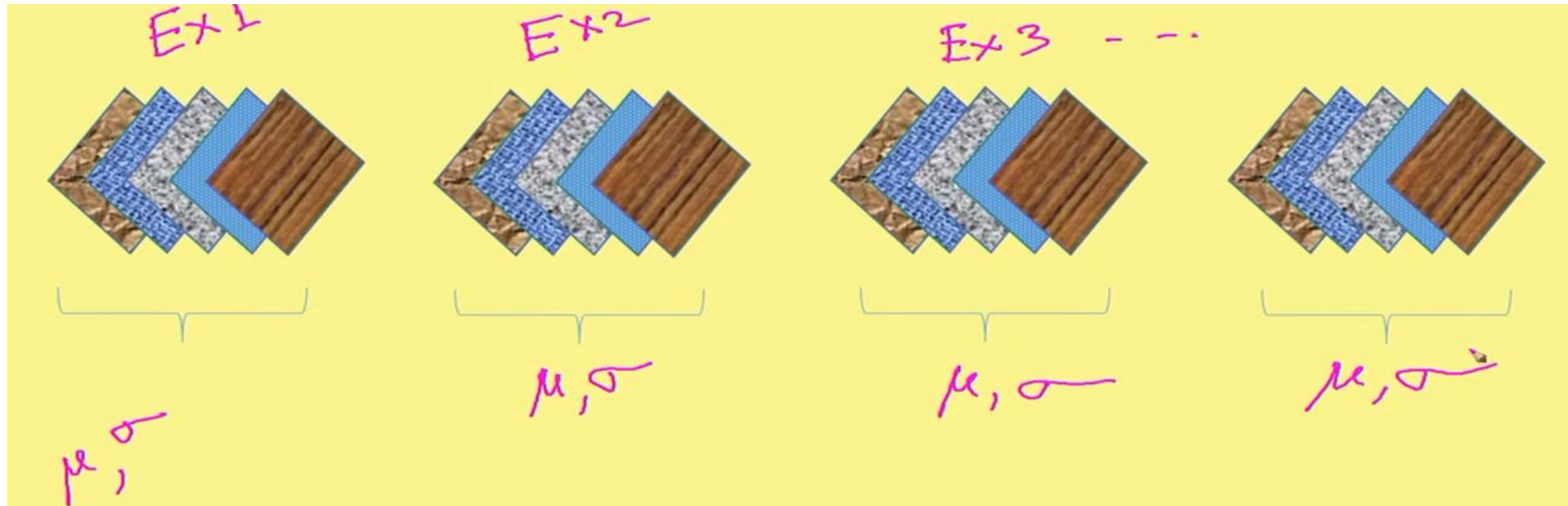
	Feature 1	Feature 2	Feature 3	Mean	Std.
Sample 1	6	3	9	6	2.45
Sample 2	7	2	4	4.33	2.05
Sample 3	8	1	5	4.67	2.87

Independent normalization across features

Normalized features

	Feature 1	Feature 2	Feature 3
Sample 1	2	-3.75	7.75
Sample 2	7.59	-2.88	1.31
Sample 3	7.89	-4.49	2.58

Layer Normalization

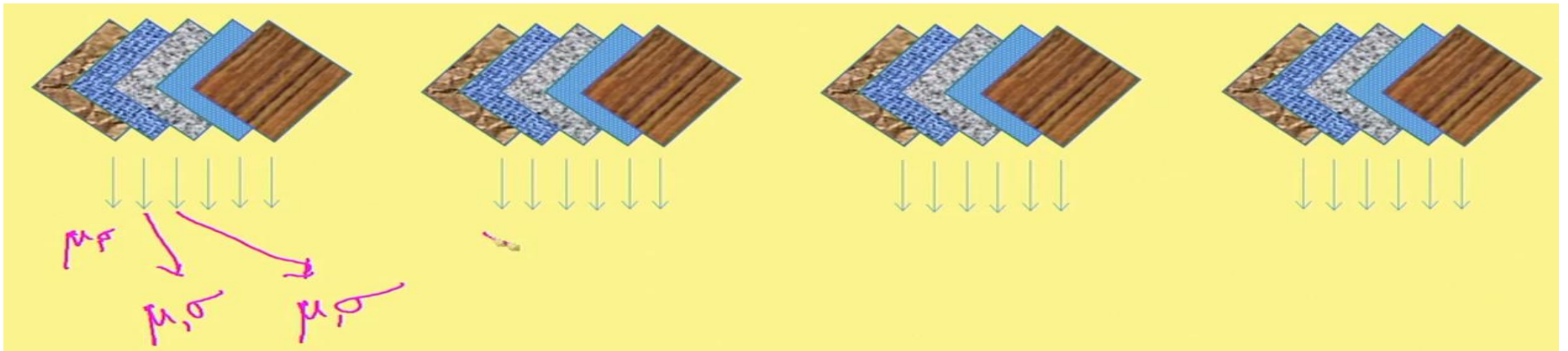


Layer Normalization

Aspect	BatchNorm	LayerNorm
Normalization Direction	Across mini-batches for each feature	Across features for each sample
Batch Size Dependence	Requires a sufficiently large batch size	Works with any batch size
Computation Scope	Per-feature across samples	Per-sample across features
Inference Behavior	Uses moving averages for mean/variance	Same as training behavior
Use Cases	CNNs, feedforward networks	RNNs, Transformers

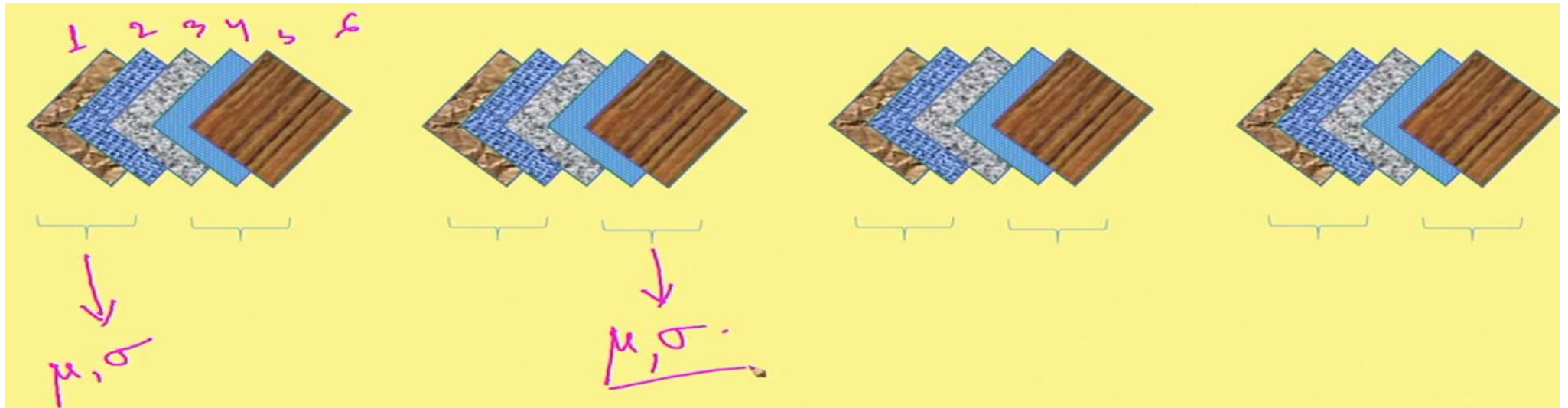
Instant Normalization

- It normalizes the activations per feature map, but independently for each instance in the batch
- It's widely used in style transfer, image generation, and artistic applications where we don't want the statistics of one image to affect another
- Used in style transfer and generative models



Group Normalization

- Group normalization operates by dividing the channels of a neural network's activations into groups and then normalizing the activations within each group.
- The normalization is done by subtracting the mean and dividing by the standard deviation of the activations within each group.

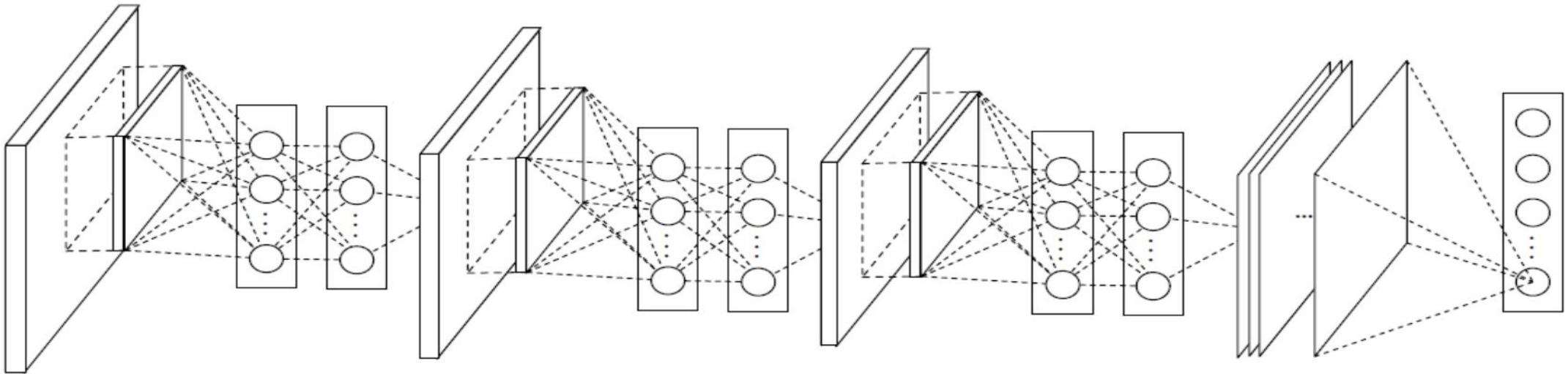


Normalization Technique	Description	Advantages	Disadvantages
Batch Normalization	Normalizes across the batch	Improves training speed and stability	Dependent on batch size
Layer Normalization	Normalizes across the features	Improves stability and robustness	Not suitable for all architectures
Group Normalization	Normalizes across groups of features	Improves stability and robustness	Requires careful group selection
Instance Normalization	Normalizes each instance independently	Improves style transfer and generative models	May not be suitable for all tasks

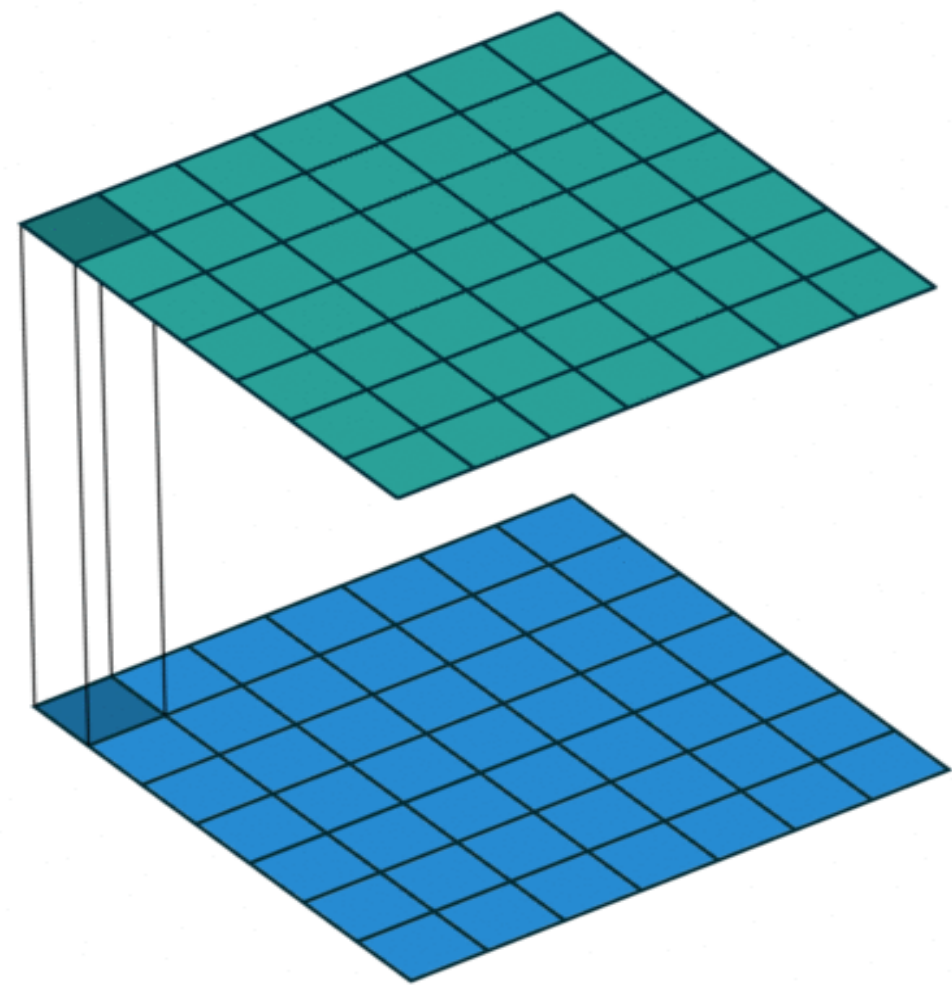
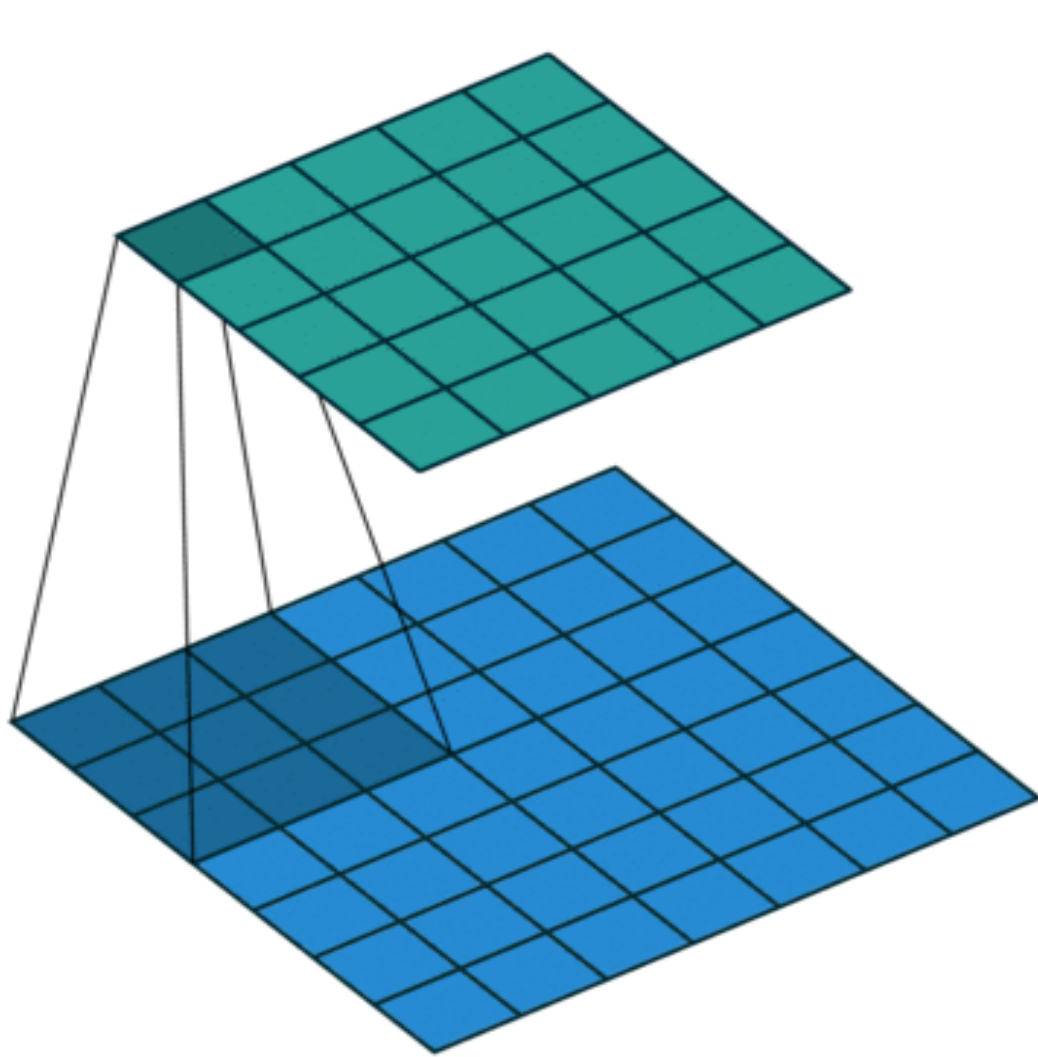
Network In Network (NiN)

Network In Network Structure

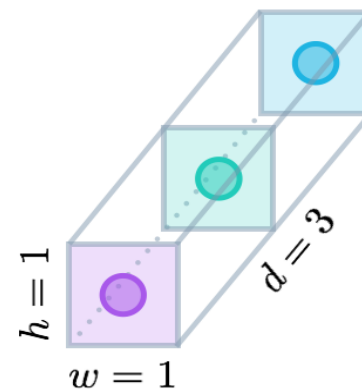
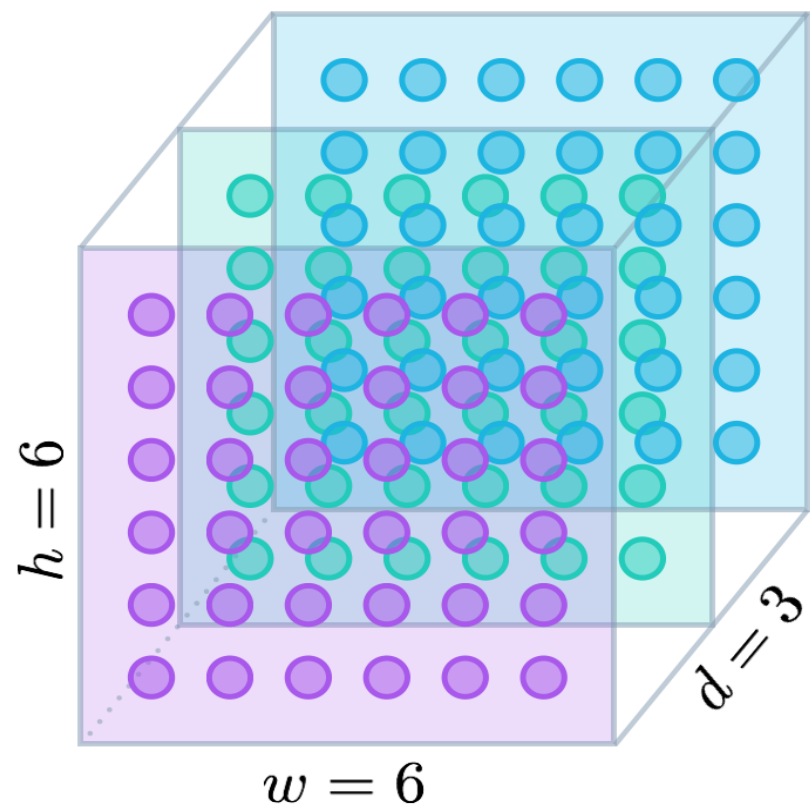
- The structure of NIN consists of stacked mlpconv layers, followed by a global average pooling layer and a final output layer.
- The number of layers in both NIN and the micro-networks can be adjusted depending on the specific task.



NINs include the stacking of three mlpconv layers and one global average pooling layer



Convolution with kernel of size 3x3 (left) vs. Convolution with kernel of size 1x1 (right)



NiN: Idea

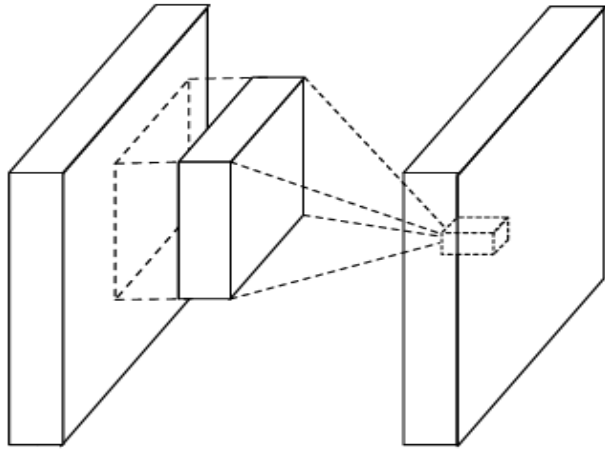
- The key idea behind NiN is to enhance the representational power of CNNs by introducing a "micro network" to process local patches of an image

OR

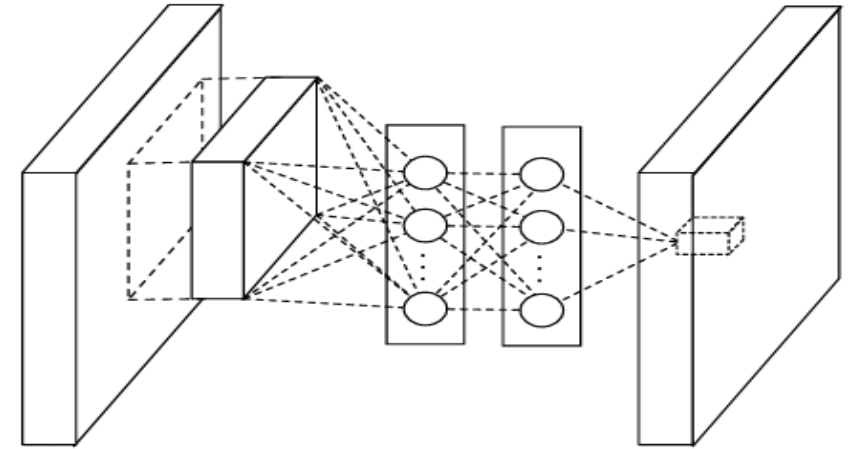
- Instead of using a traditional convolutional layer followed by a linear transformation (like in VGG or AlexNet), NiN uses a small multi-layer perceptron (micro network) inside each receptive field.

How Does NiN Work?

- NiN replaces the standard convolution layer with an Mlpconv layer, which is a series of 1×1 convolutions that are followed by non-linear activation functions.



(a) Linear convolution layer



(b) Mlpconv layer

Comparison of linear convolution layer and mlpconv layer. The linear convolution layer includes a linear filter while the mlpconv layer includes a micro network (Here, we choose the multilayer perceptron)

Architecture of NiN

Mlpconv Layers:

- Extract more complex features.
- Can perform channel-wise dimensionality reduction

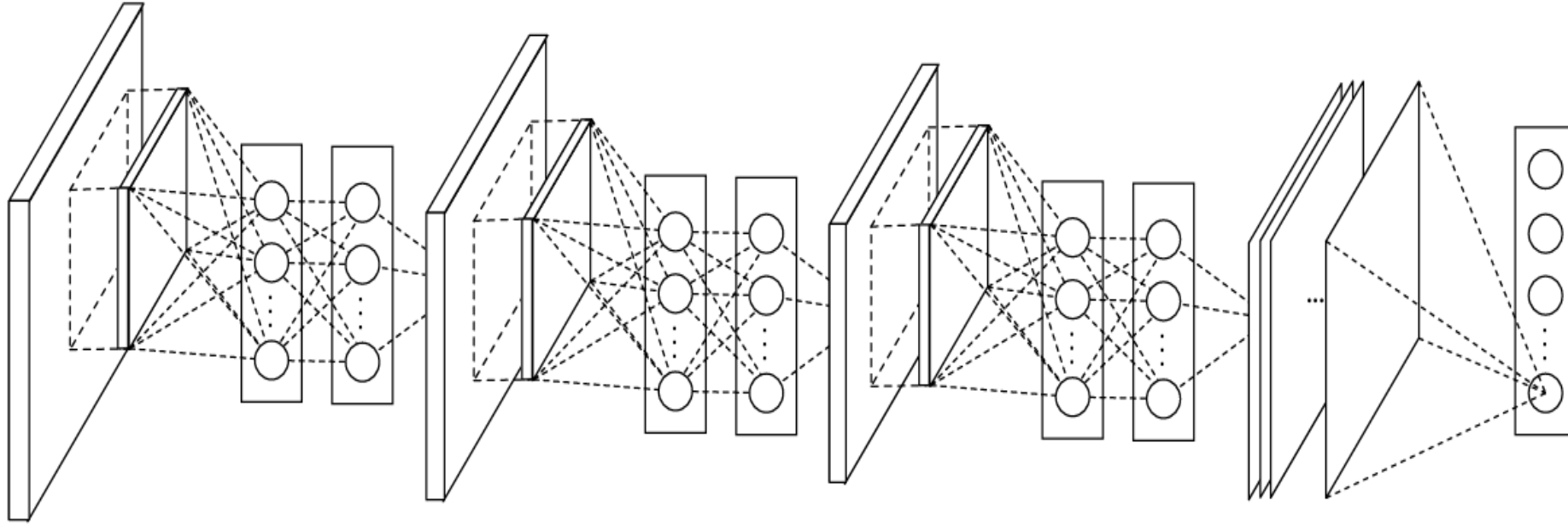
Global Average Pooling

- This reduces the risk of overfitting and
- Lowers the number of parameters.

No Fully Connected Layers:

- Makes the network more parameter-efficient.

Architecture of NiN



NINs include the stacking of three mlpconv layers and one global average pooling layer

Application

NiN itself is not widely used today as a standalone architecture, but its two major ideas became standard in deep learning

- 1×1 convolutions (mlpconv concept)
- Global average pooling

The concept of NiN is used in

- GoogLeNet
- ResNet
- DenseNet
- MobileNet
- EfficientNet